# SIH26160 — IPsec VPN Analyzer: Zero-to-Win Guide

Sep 28, 2026 · @Shreyas

This is your complete, zero-knowledge guide to SIH26160 (AI-Powered IPsec VPN Protocol Analyzer, NTRO): it teaches every concept from scratch, dissects the public competition, and lays out an architecture and 36-hour plan around one insight — most of the "AI" here is really honest protocol parsing, and the genuine machine learning is inferring what kind of traffic hides inside the encrypted tunnel.

## 1. What the problem is really asking

In one sentence: **build a tool that watches IPsec VPN traffic, works out how the VPN is configured, judges whether that configuration is secure, and — without breaking the encryption — infers what kind of activity is happening inside the tunnel, then writes a report about it.**

Think of an IPsec VPN as an armoured tunnel between two points on the internet. Your tool is a security inspector standing beside the tunnel with a notebook. The inspector cannot open the armoured cars — that would mean breaking encryption, which is infeasible — but can still learn a surprising amount by watching: which tunnel design the two sides agreed on, how thick the armour is, how often the cars pass, and how big they are.

The statement asks for five things:

1. **A testbed** — a lab where you set up IPsec VPNs in many configurations, so you have traffic to analyse. This is scored, and most teams skip it.
2. **Traffic capture** — record that traffic into `.pcap` files using tools like Wireshark or tcpdump.
3. **AI-based protocol identification** — from the captured traffic, automatically determine the IKE version, mode, encryption algorithm, authentication, key exchange, and *what type of traffic is inside the encrypted part*.
4. **Security assessment** — grade the configuration: is the crypto strong, is Perfect Forward Secrecy on, are keys rotated, is replay protection present?
5. **Reporting** — produce an executive report, a technical report, a risk score, a threat matrix, and an AI confidence score, on a dashboard.

### The one insight that wins this

Almost every competing team reads the words "AI-powered" and assumes the whole thing is a machine-learning problem. It is not, and pretending otherwise is how teams lose. The task splits into two very different halves:

- **Identifying the protocol and crypto config is mostly NOT machine learning.** The IKE negotiation (the tunnel's opening handshake) is sent in *cleartext*. The encryption algorithm, integrity algorithm, Diffie-Hellman group, and IKE version are literally written in the first packets — you read them with a parser, not a neural network. Claiming an "AI model that detects AES-256" when the packet plainly says AES-256 is a red flag any NTRO judge spots instantly.
- **The genuine machine learning is inferring the traffic type *inside* ESP.** Once the tunnel is up, the payload is encrypted and opaque. Guessing whether it carries VoIP, video, web, or email from only packet sizes and timing is a real, hard, published ML problem — and it is the one place your ML depth becomes a moat.

So the winning framing is: **be honest about which half is deterministic and which half is ML, do the deterministic half rigorously, and put your real ML effort into the encrypted-traffic inference.** Sections 7 and 8 build this out; it is the spine of the whole project.

## 2. VPNs and IPsec from zero

A **VPN (Virtual Private Network)** is a way to send data securely across an untrusted network (usually the public internet) as if the two ends were on the same private, protected wire. It does two jobs: it *encrypts* your data so eavesdroppers cannot read it, and it *authenticates* the other end so you know you are talking to the right party and the data was not tampered with.

There are several VPN technologies — OpenVPN, WireGuard, TLS-based VPNs — but this problem is specifically about **IPsec**.

### Why IPsec

**IPsec (Internet Protocol Security)** is not a single program; it is a *suite* of protocols standardised by the IETF that adds security directly at the network (IP) layer. Because it operates at the IP layer, it can protect *any* traffic above it — web, email, voice, video — transparently, without the applications knowing. That generality is why it became the default for **site-to-site VPNs** (connecting two office networks) and why it is entrenched in enterprise, government, military, and cloud infrastructure. The statement calls this out directly, which is why NTRO cares.

IPsec provides three security guarantees:

- **Confidentiality** — the data is encrypted, so outsiders cannot read it.
- **Integrity** — any tampering with the data in transit is detected.
- **Authentication** — each end proves its identity, so an attacker cannot impersonate a peer.

### Where IPsec runs (the two common deployments)

- **Site-to-site (gateway-to-gateway):** two routers/firewalls build a permanent tunnel between two networks. A person in office A reaches a server in office B, and their traffic is automatically protected across the internet in between. This is the classic IPsec use and pairs with *tunnel mode* (Section 4).
- **Remote-access (host-to-gateway):** a single laptop connects into a corporate network. Common in "IPsec VPN client" software.

### Why IPsec security is fragile (the reason this PS exists)

IPsec is only as strong as its *configuration*. The administrator chooses the encryption algorithm, the authentication method, the key-exchange group, key lifetimes, and the mode. Get any of these wrong — an outdated cipher, a weak Diffie-Hellman group, no Perfect Forward Secrecy, IKEv1 aggressive mode — and the tunnel looks secure but is not. Real deployments are full of these mistakes, and there is no easy automated way to audit them across captured traffic. That gap is exactly what SIH26160 asks you to fill.

## 3. The building blocks — AH, ESP, IKE, and the SA

IPsec is made of a few parts. You need to know four names cold, because your tool detects and reasons about all of them.

### AH — Authentication Header

AH provides integrity and authentication but **no encryption**. It proves a packet was not altered and came from the right peer, but it does not hide the contents. Because it does not encrypt, it is rarely used today, and the statement marks AH support as *optional*. Know what it is; do not spend time on it.

### ESP — Encapsulating Security Payload

ESP is the workhorse. It provides **encryption** (confidentiality) plus integrity and authentication. When an IPsec VPN is carrying real data, that data is inside **ESP packets**. This is the part your tool cannot read — and the part whose *contents you must infer* with ML. An ESP packet on the wire exposes only a few non-secret fields:

| ESP field | Visible? | Use to your tool |
|---|---|---|
| SPI (Security Parameters Index) | Yes (cleartext) | Identifies which tunnel/SA this packet belongs to; lets you group packets into flows |
| Sequence number | Yes (cleartext) | Detects replay protection; shows packet ordering |
| Encrypted payload | No (opaque) | The actual data — you never see it, only its size |
| Integrity Check Value (ICV) | Yes (bytes present) | Its presence/size hints at the auth algorithm |

Everything your ML uses — packet size, timing, direction, burst patterns — comes from the outside of ESP packets, never the inside.

### IKE — Internet Key Exchange

Before any ESP data flows, the two peers must agree on *how* to protect it: which cipher, which keys, which group. They do this with a handshake protocol called **IKE**, which runs over UDP port 500 (and port 4500 when NAT is involved). IKE is where the two sides negotiate everything — and, crucially, **the opening IKE messages are in cleartext** (Section 5). This is the gift that makes protocol identification mostly a parsing job, not an ML job.

### SA — Security Association

An **SA (Security Association)** is the agreed "contract" for one direction of protected traffic: it records the chosen algorithms, keys, key lifetime, and the SPI that labels it. IKE's whole purpose is to establish SAs. There is typically an *IKE SA* (protecting the negotiation itself) and one or more *Child SAs / IPsec SAs* (protecting the actual ESP data). When the statement says "Security Association characteristics" and "key lifetime," it is asking your tool to report properties of these SAs.

**Mental model:** IKE is the negotiation meeting that produces a signed contract (the SA); ESP is the armoured convoy that then runs under that contract; AH is a tamper-seal with no curtains; the SPI is the contract number stamped on every vehicle.

## 4. Tunnel vs Transport mode

IPsec can wrap a packet in two ways. The statement lists both as configurations you must generate and detect.

Every IP packet has two parts: a **header** (who it is from and to — source and destination IP) and a **payload** (the actual data). The difference between the two modes is *how much of the original packet gets protected and hidden*.

| | Transport mode | Tunnel mode |
|---|---|---|
| What is encrypted | Only the original payload | The **entire** original packet (header + payload) |
| Original IP header | Kept, visible | Hidden inside; a **new** outer IP header is added |
| Typical use | Host-to-host (two machines talking directly) | Site-to-site VPN (gateway-to-gateway) |
| What an observer sees | The real source and destination | Only the two gateway IPs, not the real endpoints |

**Transport mode** protects the payload but leaves the original IP header exposed, so anyone watching still sees the true endpoints. It is lighter and used when two hosts talk directly.

**Tunnel mode** takes the whole original packet, encrypts it, and wraps it in a brand-new outer packet addressed gateway-to-gateway. The real internal addresses are hidden. This is what site-to-site VPNs use, and it is the more common case.

### Why this matters for detection (an honesty point)

Here is a subtlety that separates a credible tool from a hand-wavy one: from *passive traffic alone*, reliably telling tunnel from transport mode is genuinely hard, because the mode is negotiated inside the encrypted IKE_AUTH exchange, not the cleartext part. You can make an educated inference — tunnel mode often shows gateway IPs carrying ESP for many internal hosts, and adds a fixed size overhead from the extra IP header — but you should report it *with a confidence level*, not as a certainty. Competitors who claim 100% mode detection from a passive capture are overclaiming, and NTRO judges know it. Being explicit about this limitation is a mark in your favour (Section 7 expands on what is and is not visible).

## 5. The IKE handshake — what gets negotiated

IKE is the opening handshake that sets up the tunnel. Understanding it is the single most useful thing for this project, because **most of your "protocol identification" is just reading IKE**.

## IKEv1 vs IKEv2

There are two versions, and telling them apart is one of your tool's easy wins:

- **IKEv1** — the old version (1998). It negotiates in two phases, and Phase 1 has two variants: *Main Mode* (safer) and *Aggressive Mode* (faster but insecure — it exposes a password hash that can be cracked offline). Seeing IKEv1 at all, and Aggressive Mode especially, is a security finding.
- **IKEv2** — the modern version (2005+). Simpler, faster, more robust. It is what secure deployments should use. Detecting IKEv1 vs IKEv2 is a headline output of your tool and is trivially readable from the packets.

## What actually happens in IKEv2 (the important case)

IKEv2 sets up the tunnel in two exchanges:

1. **IKE_SA_INIT** — the first exchange, sent **in cleartext**. The two peers exchange their *proposals*: the list of algorithms each supports, plus Diffie-Hellman public values and nonces. This is where your parser strikes gold (see below).
2. **IKE_AUTH** — the second exchange, now **encrypted** using keys derived from the first. Here they prove identity and set up the first Child SA. Because it is encrypted, you cannot read its contents passively.

## What is readable in cleartext from IKE_SA_INIT

This is the heart of the "identification" half of the project. Inside the cleartext IKE_SA_INIT, the Security Association payload carries the *transforms* — and these are directly visible in a tool like Wireshark:

| Field in IKE_SA_INIT | What it tells you |
|---|---|
| Encryption algorithm (transform) | e.g. AES-CBC-128, AES-GCM-256 — the cipher and key size |
| Integrity algorithm (transform) | e.g. HMAC-SHA2-256, or none (for AEAD ciphers like GCM) |
| PRF (pseudo-random function) | The key-derivation function agreed |
| Diffie-Hellman group (transform) | e.g. MODP-2048, ECP-384 — the key-exchange strength |
| Nonces, SPIs | Session identifiers; let you correlate the flow |

**This is why the "AI detects the encryption algorithm" claim is misleading:** the encryption algorithm is written, in plain text, in the proposal. You detect it by parsing the IKE Security Association payload — a deterministic job that a library like Scapy or the `pyshark` wrapper around Wireshark does directly. Your AI credibility should come from the *ESP traffic inference* (Sections 8–9), not from re-labelling something the packet already states.

> One nuance to state in your report: the proposals list what each side *offered*; the single chosen transform is confirmed in the response. Parse the responder's accepted proposal for the definitive answer, and note that if only a partial capture is available you report the negotiated set with a confidence flag.

## 6. Cryptography you must know

You do not need to be a cryptographer, but your security assessment (Section 10) grades these choices, so you must know what each one is and which values are weak. Here is everything the statement references.

### Symmetric encryption — AES and its key sizes

**AES (Advanced Encryption Standard)** is the cipher that actually scrambles the data. It comes in key sizes: **AES-128** and **AES-256** (the number is the key length in bits). Both are considered strong today; 256 is stronger and preferred for high-security/government use. Older ciphers — **DES, 3DES** — are broken or deprecated and should be flagged as weak.

### Two ways to combine encryption with integrity

Encryption hides data; you separately need *integrity* (proof it was not altered). There are two designs, and the statement names both:

- **AES-GCM** — an **AEAD** cipher (Authenticated Encryption with Associated Data). It does encryption *and* integrity in one step. Modern, efficient, preferred. When GCM is used, there is no separate HMAC.
- **AES-CBC + HMAC** — the older split design: AES in CBC mode encrypts, and a separate **HMAC** (a keyed hash, e.g. HMAC-SHA256) provides integrity. Still acceptable if the HMAC is strong, but more error-prone and slower than GCM.

### HMAC and hash strength

**HMAC** is a keyed hash used for integrity/authentication. Its strength depends on the hash: **HMAC-SHA256/384/512** are strong; **HMAC-MD5** and **HMAC-SHA1** are weak (MD5 and SHA1 are broken) and should be flagged.

## Diffie-Hellman groups (key exchange)

**Diffie-Hellman (DH)** is how the two peers agree on a shared secret key over an open channel without ever sending the key. A **DH group** specifies the size/type of the math used. Bigger and modern = stronger:

- **Weak:** MODP-768 (Group 1), MODP-1024 (Group 2), MODP-1536 (Group 5) — flag these.
- **Acceptable/Strong:** MODP-2048 (Group 14) and above, and the elliptic-curve groups **ECP-256/384** (Groups 19/20).

## Perfect Forward Secrecy (PFS)

**PFS** means each session generates a fresh, independent key via a new DH exchange, so even if one key is later compromised, past and future sessions stay safe. PFS **on** is good; PFS **off** is a finding, because one leaked key would expose more traffic. Detecting whether PFS is negotiated is one of your assessment checks.

## Key lifetime and rekeying

An SA's keys should expire and be renegotiated (**rekeyed**) periodically. Very long key lifetimes are a weakness (more data under one key = bigger blast radius if it leaks). You report the negotiated lifetime where visible.

## Anti-replay protection

ESP packets carry a **sequence number** so the receiver can reject duplicated/replayed packets (an attacker re-sending captured packets). Anti-replay **enabled** is expected; if it looks disabled, that is a finding. You can reason about this from the ESP sequence-number behaviour in the capture.

### The quick weak-vs-strong reference

| Category | Weak (flag it) | Strong (good) |
|---|---|---|
| IKE version/mode | IKEv1, Aggressive Mode | IKEv2 |
| Encryption | DES, 3DES, AES-CBC-128 (borderline) | AES-GCM-256, AES-256 |
| Integrity/hash | MD5, SHA1 | SHA-256/384/512 |
| DH group | 1, 2, 5 (768–1536-bit) | 14+ (2048-bit+), ECP-256/384 |
| PFS | Disabled | Enabled |
| Key lifetime | Very long / none | Bounded, regularly rekeyed |

These thresholds line up with strongSwan's own security recommendations and NSA/NIST guidance, which you should cite in your report to show the grading is not arbitrary.

## 7. What an analyst can actually SEE (the visibility map)

This is the most important table in the whole guide. Every design decision follows from knowing exactly what a passive observer can and cannot read. Build your tool around this reality, and state it plainly in your report — it is what makes your work credible to NTRO instead of hand-wavy.

| Property the PS asks for | Where it lives | Passively visible? | How your tool gets it |
|---|---|---|---|
| IKE version (v1/v2) | IKE header | **Yes** | Parse the IKE header — deterministic |
| Main vs Aggressive mode (v1) | IKE exchange type | **Yes** | Parse IKE exchange type — deterministic |
| Encryption algorithm | IKE_SA_INIT proposal (cleartext) | **Yes** | Parse SA transforms — deterministic |
| Integrity/auth algorithm | IKE_SA_INIT proposal (cleartext) | **Yes** | Parse SA transforms — deterministic |

| Property the PS asks for | Where it lives | Passively visible? | How your tool gets it |
|---|---|---|---|
| Key exchange (DH group) | IKE_SA_INIT proposal (cleartext) | **Yes** | Parse SA transforms — deterministic |
| PFS enabled/disabled | Child SA proposal | **Partly** | Inferred from proposals / rekey DH — report confidence |
| Tunnel vs Transport mode | IKE_AUTH (encrypted) | **No (inferred)** | Heuristics + overhead size — report confidence |
| SA parameters / key lifetime | IKE_AUTH (encrypted) | **Mostly no** | Only if offered in cleartext notify; else report as unknown |
| Replay protection | ESP sequence numbers | **Yes (behavioural)** | Analyse ESP sequence-number pattern |
| **Traffic type inside ESP** | ESP payload (encrypted) | **No — must be inferred** | **Machine learning on packet size/timing** |
| The actual data content | ESP payload (encrypted) | **Never** | Not possible — and you must say so |

## Read this table twice

Notice the pattern: **almost everything in the "identification" list is deterministic parsing of cleartext IKE.** Only two things are genuine inference — mode detection (a heuristic) and, above all, the **traffic-type-inside-ESP** row, which is the real machine-learning problem and your differentiator.

This single table is also your best defence in the Q&A round. When a judge asks "how does your AI know it is AES-256?", the winning answer is: *"It does not need AI — the algorithm is stated in the cleartext IKE proposal, which we parse deterministically and verify. We reserve machine learning for the one thing that is genuinely hidden: classifying the application inside the encrypted ESP stream from side-channel features. Here is our accuracy for that, per configuration."* That answer wins the room. Overclaiming loses it.

# 8. The two core problems

Your whole build divides into two engines with very different natures. Keeping them separate in your architecture, your slides, and your demo is what makes the project legible and honest.

## Engine A — Protocol & config identification (deterministic)

**Input:** a `.pcap` capture. **Output:** IKE version, mode, encryption/integrity/DH algorithms, SPIs, SA characteristics, and a security grade.

**How it works:** parse the packets. The IKE cleartext gives you almost everything (Section 5, 7). No training, no model — just careful protocol parsing with a library such as **Scapy**, **pyshark** (a Python wrapper over Wireshark/tshark), or **dpkt**. This engine is reliable, fast, and explainable, and it produces the bulk of the report fields. Do it thoroughly and correctly and you already beat the many teams whose parsing is shaky.

**Why call it "AI" at all?** You can add a light, honest ML touch — for example, a classifier that flags *anomalous* or *non-standard* IKE patterns the parser does not recognise, or a model that fingerprints the vendor/implementation from negotiation quirks. But do not dress up plain parsing as AI.

## Engine B — Encrypted-traffic inference (the real ML)

**Input:** the stream of ESP packets (which you cannot decrypt). **Output:** a prediction of *what kind of application traffic* is inside — VoIP, video streaming, web browsing, email, chat, file transfer — plus a confidence score.

**How it works:** you never see the payload, so you learn from the *shape* of the traffic — packet sizes, inter-arrival times, direction, burst patterns. Different applications have distinctive shapes: VoIP is small, regular, steady packets; video streaming is large bursts then pauses; web browsing is bursty and asymmetric. A machine-learning model trained on labelled examples learns these signatures. This is a well-studied field called **encrypted traffic classification**, and Section 9 is a full deep-dive.

**Why this is your moat:** it is the only part that genuinely needs ML, it is hard to do well, and it produces the "wow" demo moment — telling what is inside an encrypted tunnel without decrypting it. Most competitors under-build this and over-build the parsing. You do the opposite.

## How the two engines combine

The final report fuses both: Engine A says *"IKEv2, AES-GCM-256, DH group 14, PFS on — strong config"*; Engine B says *"the traffic inside looks like video streaming, 87% confidence"*; the assessment layer (Section 10) turns Engine A's findings into a risk score

and threat matrix. One capture in, one graded intelligence report out.

## 9. Deep dive: encrypted-traffic classification

This is Engine B, your differentiator. The goal: given a flow of encrypted packets, predict the application category inside — without decryption. Here is how the field actually does it.

### The core idea: learn from shape, not content

Even when payload is encrypted, the *metadata* leaks behaviour. Each application produces a characteristic pattern:

- **VoIP (voice call):** small packets (~60–200 bytes), very regular timing (every ~20 ms), symmetric both directions, steady — like a metronome.
- **Video streaming:** large packets, big bursts followed by pauses (buffer fill then wait), strongly download-heavy.
- **Web browsing:** bursty and irregular — a flurry when a page loads, then idle; asymmetric (small requests, larger responses).
- **Email / file transfer:** longer sustained transfers, mostly one direction.
- **Chat / messaging:** small, sparse, irregular packets.

A model that has seen many labelled examples learns to tell these apart from the pattern alone.

### The features (what you actually feed the model)

You compute *statistical features* per flow (a flow = packets sharing the same SPI / IP pair). The standard feature families, used by tools like **CICFlowMeter**:

- **Packet size stats:** min, max, mean, standard deviation of packet lengths; separately for forward and backward directions.
- **Inter-arrival time (IAT) stats:** min/max/mean/std of the gaps between packets — the timing fingerprint.
- **Flow rate:** bytes per second, packets per second.
- **Directional ratios:** forward vs backward byte and packet counts (asymmetry).
- **Burstiness:** active/idle periods, flow duration.

For IPsec ESP specifically, you extract these from the ESP packet headers (size, time, direction, SPI) — all visible without decryption.

### The models (from simplest to strongest)

1. **Classical ML on flow features** — Random Forest, XGBoost, or an SVM over the CICFlowMeter-style features. This is the pragmatic winner: fast to train, easy to explain, and strong. Published work on the ISCXVPN2016 dataset reaches ~98% accuracy with a stacking ensemble (RF + NN + SVM), and Random Forest / KNN alone reach ~96–97%.
2. **Deep learning on raw sequences** — a 1D-CNN or LSTM fed the sequence of packet sizes/timings directly, learning features itself. Higher ceiling, more data and tuning needed. Good as a "stretch" model to mention.
3. **Honest abstention** — whatever model you pick, output an **UNKNOWN / low-confidence** class rather than forcing a guess. Judges respect a tool that says "I am not sure" over one that confidently mislabels.

### Reporting it honestly (this wins marks)

Accuracy is **not uniform across configurations**, and saying so is a strength:

- **AES-CBC** leaves packet sizes in coarse, predictable blocks (padding to 16-byte multiples) — easier to fingerprint.
- **AES-GCM** leaks slightly less structure — harder.
- **Padding / traffic shaping** deliberately hides patterns — accuracy drops, and you should show that honestly.

So your result is not one number but a small matrix: *accuracy per traffic type per cipher config*. That nuance is exactly what an NTRO evaluator wants to see, and almost no competitor will present it.

### The order of difficulty (set expectations)

Distinguishing coarse categories (VoIP vs video vs bulk) is quite achievable. Fine-grained app identification (which specific app) is much harder and you should treat it as a stretch goal, not a promise.

## 10. The security assessment engine

Once Engine A has extracted the configuration, this layer grades it. It is a **rules engine**: each detected setting is checked against a policy of known-good vs known-weak values (Section 6), each failure gets a severity, and the severities roll up into an overall risk score and a threat matrix. This is deterministic, explainable, and easy to defend — exactly what a security tool should be.

## How scoring works

1. Each check produces **pass / fail** plus a **severity** (Critical / High / Medium / Low) and a CVSS-style rationale.
2. Map severities to weights (e.g. Critical = 40, High = 20, Medium = 10, Low = 5).
3. Overall risk score = start at 100, subtract the weighted failures, and bucket into a rating (e.g. 90–100 Strong, 70–89 Moderate, below 70 Weak).
4. Attach a **remediation** to every failure — the exact stronger setting to use. This is high-value and easy.

## The threat matrix (a worked example)

This is the kind of table your report generates per analysed tunnel:

| Check | Detected | Verdict | Severity | Remediation |
|---|---|---|---|---|
| IKE version | IKEv1 (Aggressive) | Fail | Critical | Migrate to IKEv2; disable Aggressive Mode |
| Encryption | 3DES | Fail | High | Use AES-GCM-256 |
| Integrity | HMAC-SHA1 | Fail | High | Use SHA-256 or above |
| DH group | Group 2 (MODP-1024) | Fail | High | Use Group 14+ or ECP-384 |
| PFS | Disabled | Fail | Medium | Enable PFS on the Child SA |
| Key lifetime | 24 h | Warn | Low | Shorten and rekey more often |
| Anti-replay | Enabled | Pass | — | None |

The example above is a deliberately bad tunnel; a good tunnel would show mostly passes. Your report renders one such matrix per capture, plus the executive summary (a one-line verdict and score for a non-technical reader) and the technical report (the full detail with packet evidence).

## What "AI confidence score" means here

The statement asks for an *AI Confidence Score*. Be precise about what it applies to: it is the confidence of **Engine B's traffic-type prediction** (a probabilistic model output), not of the deterministic config checks (which are certain when the IKE cleartext is present). Reporting confidence only where it genuinely applies is another credibility signal.

### Grounding the policy (cite it)

Do not invent thresholds. Base your pass/fail policy on published standards and cite them in the report: **strongSwan security recommendations**, **NIST SP 800-77 / 800-52**, the **NSA guidance on configuring IPsec VPNs**, and CIS-style benchmarks. A judge seeing sourced thresholds trusts the grading; a judge seeing arbitrary numbers does not.

## 11. Competitor analysis

The statement had 36 idea submissions when reviewed, and several teams have public GitHub repos already. This is good news: you can see exactly what the field looks like and where the gaps are. Below is what the serious public entries do, and how you beat them.

| Repo / team | What they built | Strengths | Gaps you exploit |
|---|---|---|---|
| Alchemist (naman9271) | Full-stack passive `.pcap` analyzer; Next.js + Go + Python; own "IPsec PCAP Lab" dataset; LLM report explanations; live deploy | Most complete rival; honest abstention; clean scope; multi-layer architecture | ML is supplementary, not the core; no testbed-generation deliverable; no per-config accuracy matrix; can't verify mode/SA without telemetry |
| TunnelScope (shivanshu1512) | Passive analyzer; infers ESP cipher mode from packet-length residues; parses IKEv2 cleartext; risk scoring; ML module + frontend | Clever packet-length-residue idea; right instincts on cleartext parsing | Early stage (few commits); thin ML; no rigorous dataset/metrics; no automated testbed |
| Others (CipherGuard, IPsec_Sentinal_AI, IPsecGuard-AI, etc.) | Early-stage analyzers, mostly parsing + dashboard | Show the common template | Shallow; little/no real ML; no testbed; no honest metrics |

### The five gaps that win

Across the whole field, the same weaknesses repeat. Hit all five and you are the strongest entry:

1. **Nobody builds the testbed well.** Part (a) is a *scored deliverable*, and most teams only analyse pre-made pcaps. An automated testbed that spins up IPsec tunnels in every listed configuration and auto-generates a labelled dataset is a visible, defensible advantage — and it feeds your ML.
2. **The ML is an afterthought for everyone.** The one genuinely hard, genuinely-AI part (ESP traffic inference) is where rivals are weakest. Make it your core and report real numbers.
3. **No one reports honest per-config accuracy.** A small accuracy matrix (traffic type × cipher) signals rigour and beats a single inflated number.
4. **Overclaiming on hidden fields.** Rivals claim certainty on mode/SA details that are actually encrypted. Your confidence-flagged honesty is a differentiator with NTRO judges.
5. **Weak grounding.** Few cite standards for their pass/fail thresholds. Sourced policy (strongSwan/NIST/NSA) makes your grading trustworthy.

### Your one-line positioning

*"A rigorous, honest IPsec analyzer: deterministic parsing for everything the protocol reveals, real machine learning for the one thing it hides (the traffic inside ESP), an automated testbed that proves our numbers, and every verdict graded against published standards — with confidence scores where, and only where, they belong."*

That combines the best of every competitor (Alchemist's honesty and full-stack polish, TunnelScope's cleartext parsing and residue idea) and adds the testbed, the real ML, and the metrics they lack.

## 12. Your winning architecture

The system is a straight pipeline: a capture goes in, and a graded intelligence report comes out. Keep the components cleanly separated — it makes the build parallelisable across your six people and the demo easy to narrate.

### The pipeline (data flow, in order)

1. **Testbed & capture** — automated IPsec labs (strongSwan/Libreswan) generate tunnels in every config; tcpdump records labelled `.pcap` files. (Feeds both the dataset and live demo.)
2. **Ingestion** — accept an uploaded `.pcap` / `.pcapng` (or a live interface); normalise, split into flows by SPI / IP-pair.
3. **Engine A — protocol parser** — parse IKE cleartext with Scapy/pyshark; extract version, mode, transforms (enc/integrity/DH), SPIs, SA hints, ESP sequence behaviour.

4. **Engine B — traffic classifier** — compute flow features (CICFlowMeter-style) over ESP packets; run the trained model; output traffic-type + confidence, or UNKNOWN.
5. **Assessment engine** — apply the standards-based rules to Engine A's config; produce pass/fail, severity, risk score, threat matrix, remediation.
6. **Report generator** — assemble executive report, technical report, risk score, threat matrix, AI confidence; render PDF (ReportLab/WeasyPrint).
7. **Dashboard** — web UI to upload a capture, watch analysis, and explore results interactively.

## Component / tech-stack table

| Layer | Job | Suggested tech |
|---|---|---|
| Testbed | Generate tunnels + labelled data | strongSwan / Libreswan on Linux VMs or Docker; shell/Ansible scripts |
| Capture | Record traffic | tcpdump, tshark |
| Parser (Engine A) | Read IKE/ESP metadata | Python + Scapy / pyshark / dpkt |
| Feature extraction | Flow stats for ML | CICFlowMeter or custom Python (pandas/numpy) |
| Classifier (Engine B) | Traffic-type inference | scikit-learn (RandomForest/XGBoost); optional 1D-CNN in PyTorch |
| Assessment | Grade config | Python rules engine (YAML policy) |
| Reporting | PDF + scores | ReportLab / WeasyPrint |
| Dashboard | UI | FastAPI backend + React/Next.js frontend |
| Optional LLM | Plain-language report text | a local/hosted LLM to phrase findings (label it clearly as explanation, not detection) |

## Design principles that score points

- **Offline / air-gapped capable.** Security tools for NTRO should run without phoning home. Bundle models locally.
- **Explainable.** Every finding traces to a packet or a rule. No black boxes in the deterministic path.
- **Confidence where it belongs.** Certainty for parsed facts; probability for ML inferences.

- **Modular.** Each engine runs standalone, so you can demo parts even if one breaks, and split work across the team.

*(This section is a pipeline; if you want, I can turn it into a proper block diagram in the doc as a follow-up.)*

## 13. Data and testbed

Everything here is free and needs no special hardware — which is exactly why this problem suits your constraints. You need data for two things: to *train* Engine B, and to *demo* the whole pipeline.

### Public datasets (to bootstrap the ML)

- **ISCXVPN2016 (VPN-nonVPN)** — the standard benchmark for encrypted-traffic classification, from the Canadian Institute for Cybersecurity. Labelled categories (VoIP, video, chat, email, file transfer, browsing), VPN and non-VPN. Start here.
- **VNAT** — MIT Lincoln Laboratory's VPN/non-VPN application traffic dataset; another labelled source.
- **A note on realism:** these are general VPN datasets, not pure IPsec-ESP. Use them to prototype and pre-train, then validate on your own IPsec captures (below). Say this clearly in your report — it is honest and correct.

### Build your own IPsec testbed (the winning move)

This is Part (a) of the statement and most teams skip it. It is very doable for free:

1. **Two endpoints.** Two Linux VMs (VirtualBox/VMware) or two Docker containers, or two small cloud VMs. Call them Gateway A and Gateway B.
2. **IPsec software.** Install **strongSwan** (or Libreswan) on both. It is the standard open-source IPsec implementation and supports every option the statement lists.
3. **Script the configurations.** Write a script that stands up the tunnel in each variation the statement names: Tunnel/Transport, AES-128/256, GCM vs CBC+HMAC, different DH groups, PFS on/off, IPv4/IPv6. strongSwan's config makes each a few lines, so you can loop over them.
4. **Generate labelled traffic.** Through each tunnel, run known applications and label the capture: a VoIP call (e.g. a SIP/RTP generator or a real call), video (stream a clip), web (curl/browser), email (SMTP), ICMP (ping), file transfer. You know the label because you generated it.
5. **Capture.** Run tcpdump on the gateway interface to record the ESP (and IKE) packets into per-run `.pcap` files, named by their config + traffic label.

6. **Assemble the dataset.** Extract flow features from each capture and attach the labels. This is your training and evaluation set — and, because you built it, you can prove your accuracy numbers.

### Why this doubles as your best evidence

A judge asking "how do you know your classifier works?" gets the strongest possible answer: *"We generated ground-truth IPsec traffic ourselves across every configuration, so we know exactly what is inside each tunnel and can measure our accuracy against it."* No competitor relying only on borrowed pcaps can say that. The testbed is simultaneously a scored deliverable, your data source, your evaluation harness, and your credibility.

## 14. The build-and-win plan

### Roles across your six people

- **Testbed + capture** (1) — strongSwan configs, traffic generation, dataset.
- **Engine A parser** (1) — IKE/ESP parsing, config extraction.
- **Engine B ML** (1–2) — features, model, accuracy matrix.
- **Assessment + reporting** (1) — rules engine, PDF, scoring.
- **Dashboard + integration** (1) — UI, glue, demo polish.

Every member must be able to explain their module — judges ask each person in the finale.

### Before the finale (the real work happens here)

1. Stand up the testbed and generate the labelled dataset across all configs.
2. Build and validate Engine A (parsing) — it should be near-complete; it is deterministic.
3. Train Engine B and produce the honest per-config accuracy matrix.
4. Build the assessment rules from cited standards.
5. Wire the dashboard and PDF reporting; rehearse the end-to-end demo.
6. Prepare the 5-slide submission (below).

### During the 36 hours

The finale rewards *visible progress between evaluation rounds*, so plan deltas, not one big reveal:

- **Round 1:** end-to-end demo working on a pre-built capture; lead with the ESP inference "wow".

- **Round 2:** live capture from the testbed analysed on stage; show the accuracy matrix.
- **Round 3:** hardening — edge cases, an adversarial config (traffic shaping), and honest limitations.
- Keep a working build at every checkpoint; a running ugly demo beats a broken polished one.

## The 5-slide idea submission (what evaluators score)

The statement asks for a 5-slide technical PPT. Map them to what screening rewards:

1. **Problem + insight** — the two-engine framing; name the deterministic vs ML split.
2. **Architecture** — the pipeline diagram; named technologies (Scapy, scikit-learn, strongSwan), not "AI/ML layer".
3. **The ML core** — encrypted-traffic classification: features, model, expected per-config accuracy.
4. **Testbed + data + security assessment** — your standards-based grading and the self-generated dataset.
5. **Feasibility + differentiation** — 36-hour plan, why you beat the field (the five gaps), cited sources.

## The demo script (60 seconds that win the room)

*"Here is a live IPsec tunnel we set up. Watch: our tool reads the handshake and tells us it is IKEv2, AES-GCM-256, DH group 14, PFS on — a strong config, graded against NIST. Now the hard part: the data is fully encrypted, but from packet timing and size alone our model says the traffic inside is a video stream, 89% confidence. We verify — it is. One capture in, a full graded intelligence report out."*

## Three ways teams lose this (avoid them)

- **Overclaiming** certainty on encrypted fields — judges probe and it collapses.
- **A pretty dashboard over a weak core** — invert it; depth first.
- **A single inflated accuracy number** — the honest matrix beats it every time.

# 15. Glossary and learning resources

## Every term in one place

| Term | Plain meaning |
|---|---|
| IPsec | A suite of protocols that secures traffic at the IP layer |
| VPN | Secure private connection over a public network |
| AH | Authentication Header — integrity only, no encryption (rarely used) |
| ESP | Encapsulating Security Payload — the encrypted data carrier |
| IKE | Internet Key Exchange — the setup handshake (IKEv1 / IKEv2) |
| IKE_SA_INIT | First IKEv2 exchange, in cleartext — where you read the crypto |
| IKE_AUTH | Second IKEv2 exchange, encrypted — identity + first Child SA |
| SA | Security Association — the agreed "contract" of algorithms/keys |
| SPI | Security Parameters Index — the ID stamped on each packet's tunnel |
| Tunnel mode | Encrypts the whole packet; adds a new outer header (site-to-site) |
| Transport mode | Encrypts only the payload; keeps the original header |
| AES | The symmetric cipher that encrypts the data (128/256-bit) |
| GCM | AES mode doing encryption + integrity together (AEAD) |
| CBC + HMAC | Older split: AES-CBC encrypts, HMAC checks integrity |
| HMAC | Keyed hash for integrity (SHA-256 strong; MD5/SHA1 weak) |

| Term | Plain meaning |
|---|---|
| Diffie-Hellman (DH) | Method to agree a shared key over an open channel |
| DH group | The strength/size of that key exchange (14+ good; 1/2/5 weak) |
| PFS | Perfect Forward Secrecy — fresh key per session |
| Key lifetime | How long before keys are renegotiated (rekeyed) |
| Anti-replay | Rejecting duplicated/replayed packets via sequence numbers |
| Flow | Packets sharing an SPI / IP pair — the unit your ML classifies |
| Encrypted traffic classification | Inferring the app inside encrypted traffic from size/timing |
| pcap | A packet-capture file |

## Where to learn each piece (free)

- **IPsec/IKE mechanics:** the strongSwan documentation, and Cloudflare/other "what is IPsec" primers.
- **Reading IKE in Wireshark:** the F5 "Understanding IPsec IKEv2 negotiation on Wireshark" walkthrough (in Sources) — do this hands-on early; it makes Engine A obvious.
- **Encrypted traffic classification:** the ISCXVPN2016 dataset page and its companion papers; the CICFlowMeter repo for features.
- **Security policy/thresholds:** strongSwan security recommendations and the NSA IPsec VPN configuration guide.
- **Testbed:** any "strongSwan site-to-site" tutorial (in Sources) plus the strongSwan config docs.

## First three things to do this week

1. Install strongSwan on two VMs and bring up one tunnel; capture it; open it in Wireshark and find the IKE proposal. This makes the whole project concrete in an afternoon.
2. Download ISCXVPN2016 and train a quick Random Forest on flow features to see the ML work end-to-end.
3. Draft the 5 slides around the two-engine story so the team shares one mental model.

# 16. Sources

### IPsec / IKE mechanics and visibility

- Understanding IPsec IKEv2 negotiation on Wireshark (F5) — confirms which fields are cleartext in IKE_SA_INIT vs encrypted in IKE_AUTH.
- strongSwan security recommendations — weak vs strong algorithms, DH groups, PFS, IKEv1 aggressive-mode warning.
- NSA — Configuring IPsec VPNs — government hardening guidance for your assessment policy.

### Encrypted-traffic classification (Engine B)

- ISCXVPN2016 (VPN-nonVPN) dataset — the standard labelled benchmark.
- VNAT dataset (MIT Lincoln Laboratory) — another labelled VPN/non-VPN source.
- Ensemble learning for VPN traffic classification (ScienceDirect) — ~98% with a stacking ensemble on ISCXVPN2016; feature and model reference.
- CICFlowMeter — the flow-feature extractor to base Engine B's features on.

### Testbed

- strongSwan on GitHub and configuration quickstart — build your IPsec lab.
- Site-to-site IPsec with strongSwan (tutorial) — a worked setup.

### Competitor repos (SIH26160)

- Alchemist — naman9271
- TunnelScope — shivanshu1512
- CipherGuard and other early-stage entries.

*As of 28 September 2026. Repo maturity and idea counts change — re-check the competitor repos before the finale, since some will develop further.*